

Nome: William Guilherme Prochnow
UC: Inteligência Artificial

Data: 11/12/2024

Projeto Final A3: Desenvolvimento de um Agente Inteligente

Sumário

O Que Vai Ser Resolvido?	3
Importação das Bibliotecas e Verificações Iniciais	3
• Bibliotecas	3
• Dados	3
Feature Engineering	4
• Verificação e Limpeza de Valores Faltantes	6
• Ajuste de Colunas de Texto Para Numérico	7
Separação - Treino e Teste	8
Predições e Gráficos	8
Métricas de Desempenho	10
Implementação	12
Considerações Finais e Conclusão	12
Referências Bibliográficas	14

O Que Vai Ser Resolvido?

A criação desse modelo tem o intuito em prever o resultado de um jogo do campeonato Brasileiro da série “A”, com relação ao time da casa, retornando somente um, dentre os três possíveis valores:

- **Derrota:** Quando o time da casa tem uma probabilidade maior de perder para o adversário.
- **Empate:** Quando ambos os times (casa e fora) tem uma probabilidade maior de ficarem com o mesmo placar.
- **Vitória:** Quando o time da casa tem uma probabilidade maior de vencer do adversário.

Nesse caso, o problema se encaixa em classificação, onde procuro especificar com o uso dos dados, em qual classe que determinado jogo vai se encaixar com base no problema que quero resolver.

Importação das Bibliotecas e Verificações Iniciais

É realizado o import das Libs necessárias e logo em seguida a leitura da base onde os dados estarão disponíveis.

- **Bibliotecas:**

No total, foram utilizadas cinco diferentes bibliotecas, sendo elas:

1. **Pandas:** Necessária para realizar o carregamento, a análise exploratória e alguns tratamentos dos dados;
2. **Numpy:** Usada para cálculos mais complexos envolvendo matrizes e álgebra linear;
3. **Seaborn:** Oferece uma série de gráficos estatísticos e de visualização que permitem que os usuários explorem e descubram padrões nos dados de forma rápida e intuitiva;
4. **Matplotlib:** Permite criar gráficos e visualizações de alta qualidade de forma simples e eficiente, permitindo a criação de gráficos de linha, gráficos de barra, gráficos de dispersão, histogramas, entre outros tipos de gráficos;
5. **Sklearn:** Integrada com diversos modelos matemáticos com o intuito da criação de previsões, ela é uma das principais quando procuramos realizar esse tipo de tarefa.

- **Dados:**

São originários de diversas localidades, incluindo sites e APIs, onde se encontram informações do campeonato Brasileiro da série A, valores

envolvendo os jogadores e seus clubes e também informações sobre o clima.

Demonstro uma pequena quantidade de dados com uma amostra das primeiras cinco linhas e também a fim de entender o tamanho de toda a base procuro me informar sobre a quantidade de linhas e colunas contidas ali, procuro saber os tipos de dados em cada coluna para ter uma visão do que vou ter de fazer nos tratamentos prévios para a implementação do modelo, e por último, seria uma prévia geral do que está contido em cada coluna mostrando a quantia de dados que não estão vazios, valores mínimos e máximos, etc.

Feature Engineering

Nessa parte é feito uma seleção de colunas que vão ter a melhor orientação para o modelo predizer a coluna target, que nesse caso será a “resultado_casa”. Vale destacar que essa seleção foi em cima apenas das colunas criadas com a ingestão de dados feita através do site oficial da CBF (Confederação Brasileira de Futebol), pois assim, questões relacionadas à verificação de qualidade dos dados e a confiabilidade seriam menos necessárias para se seguir na criação do modelo final.

Foram feitos dois processos de feature engineering, um deles com todas as colunas, e outro, seguindo os resultados desse primeiro, selecionando apenas as colunas que obtiveram as melhores correlações entre elas ou entre a coluna target, esses resultados podem ser observados pelos seguintes gráficos que deixo logo abaixo:

Imagem da Correlação Entre Todas as Colunas:

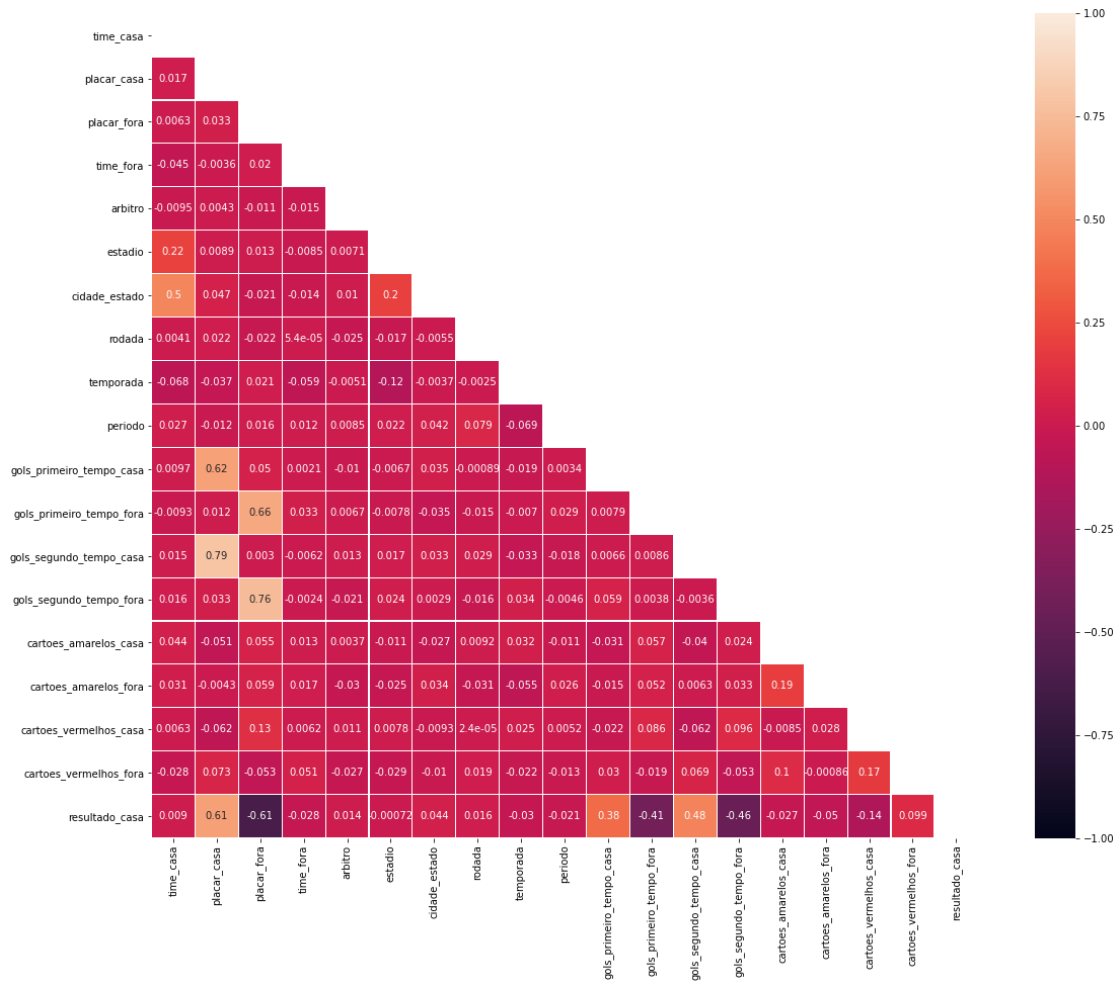
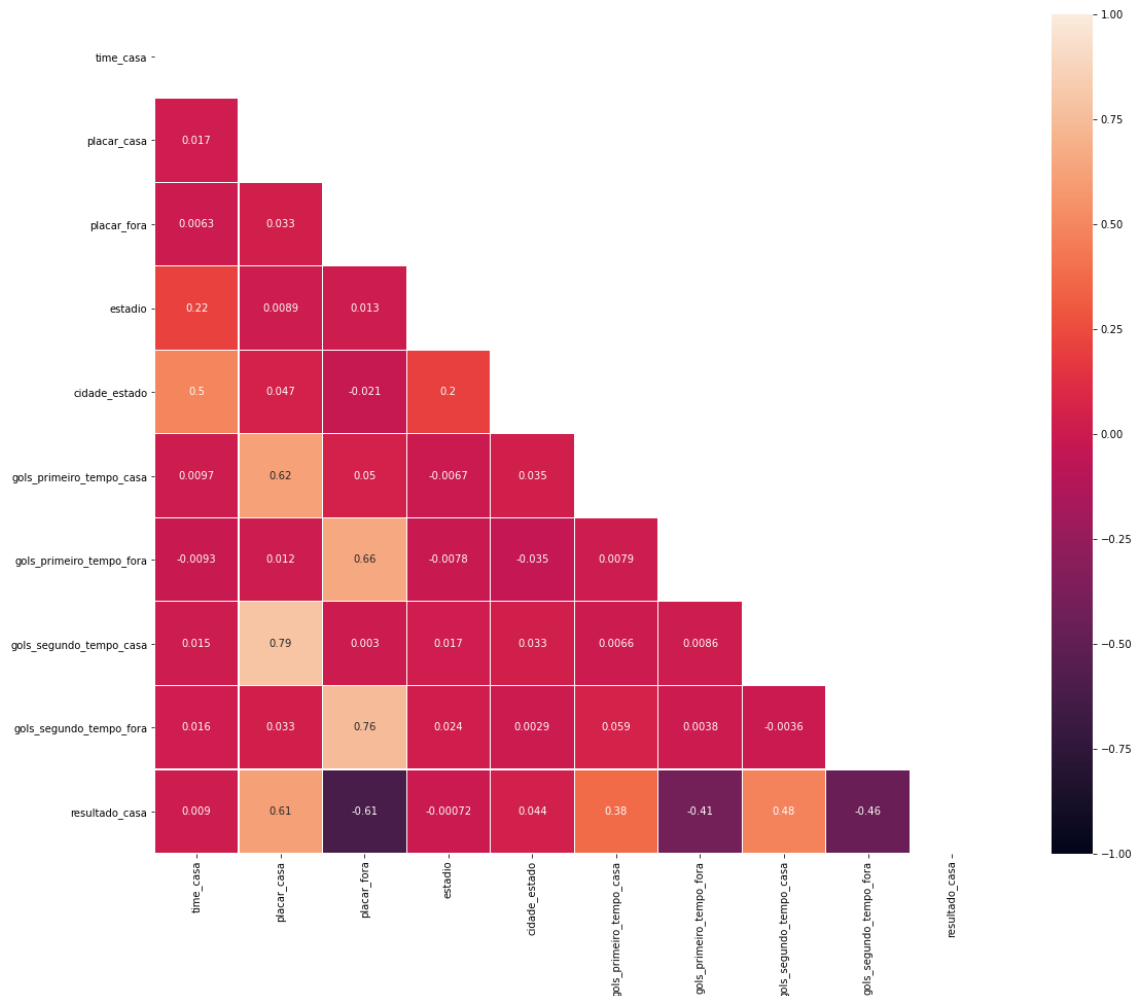


Imagem da Correlação Entre as Melhores Colunas:



Lembrando que essas colunas foram escolhidas por sua pontuação na correlação, não importando se essa pontuação for negativa ou positiva. Assim sendo, as colunas removidas foram: time_fora, arbitro, rodada, temporada, período, cartoes_amarelos_casa, cartoes_amarelos_fora, cartoes_vermelhos_casa e cartoes_vermelhos_fora.

Para se chegar nesses gráficos, esses processos foram divididos em algumas etapas como:

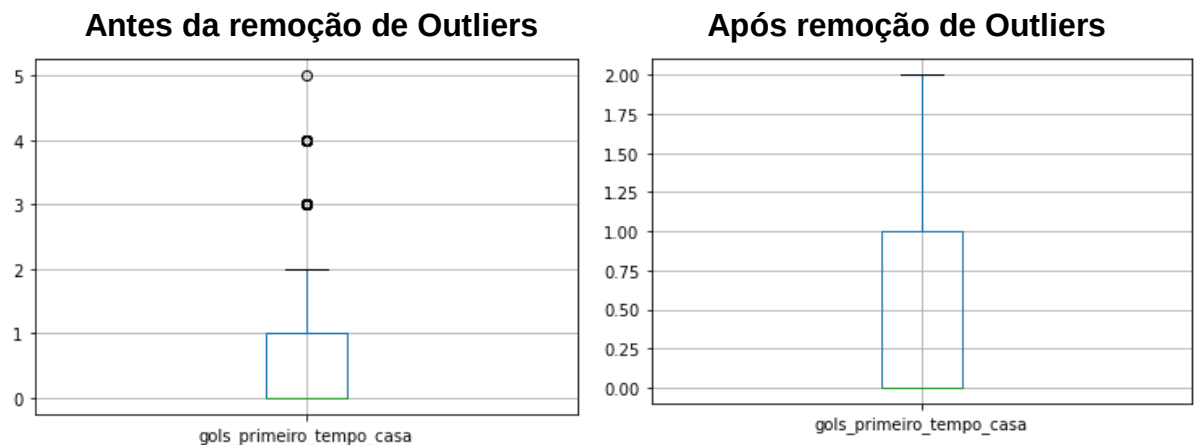
- **Verificação e Limpeza de Valores Faltantes:**

Aqui é feita a verificação das colunas com valores faltantes, e em seguida, a validação de valores que não representam tão bem a maior parte dos dados, aqueles considerados como outliers, que prejudicam o treinamento do modelo final.

Para o tratamento desses valores, é utilizada uma técnica que verifica onde se concentra a maior porcentagem de dados (chamada de percentil), para realizar esta verificação, foi pego a coluna “gols_primeiro_tempo_casa”, que contém os dados de quantos gols no primeiro tempo o time da casa realizou, e faz a remoção das linhas onde

encontra nessa coluna valores mais distantes do que se comparados com a maior parte do restante dos dados nessa coluna. E com esse tratamento, já que as linhas com valores faltantes não eram a maioria, grande parte dessas linhas também já foram removidas e nesse momento só foi colocado um pequeno filtro para retirar as últimas linhas que restaram e ainda constavam com valores faltantes.

O resultado de todo esse tratamento pode ser visualizado através dos seguintes gráficos:



Observe que na figura da esquerda a quantidade de gols varia, onde a maior parte se concentra entre 0 e 2, alguns casos chegam a alcançar 5, enquanto na figura da direita os valores ficam concentrados somente entre 0 e 2, onde a maior parte fica em 1 gol.

- **Ajuste de Colunas de Texto Para Numérico:**

Nessa etapa foi realizada a alteração de valores textuais para valores numéricos das seguintes colunas: `time_casa`, `estadio`, `cidade_estado` e `resultado_casa`, para melhor adequação dos dados no modelo.

Separação - Treino e Teste

Até então, estava tratando e selecionando as devidas colunas que mais acrescentariam ao modelo final, a partir delas, irei dividir em duas diferentes tabelas, entre quais servirão para o modelo estudar, da qual servirá para o modelo prever. Em outras palavras seriam as características (features) para predição e a coluna para ser categorizada (target) pelo modelo.

É feita a separação dos dados de treino e teste com base na separação das colunas de “feature” e “target”, utilizando 30% dos dados para teste e o restante para treino do modelo, o parâmetro “random_state” nesta separação foi fixado em 42 (significando que essa separação ocorrerá da mesma forma nas diversas vezes que rodar novamente, o que auxilia no momento de comparar os resultados finais), no final deste processo, formou-se as seguintes tabelas:

- **Features:** time_casa, placar_casa, placar_fora, estadio, cidade_estado, gols_primeiro_tempo_casa, gols_primeiro_tempo_fora, gols_segundo_tempo_casa e gols_segundo_tempo_fora.
- **Target:** resultado_casa.

Predições e Gráficos

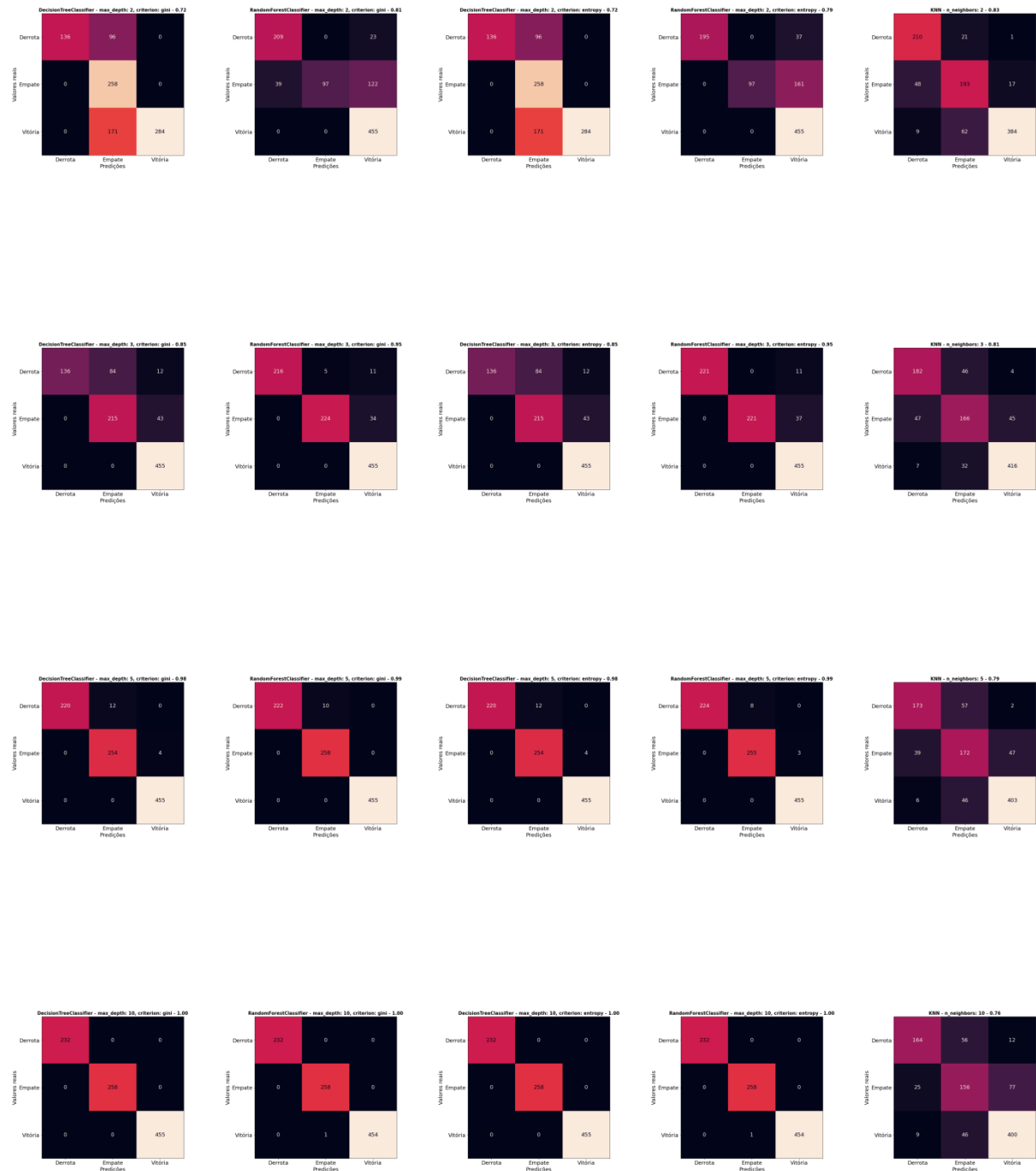
Finalmente na utilização e testes de modelos, é nessa etapa que irei colocar tudo que fiz até aqui a prova. Pois agora vou utilizar os dados que separei de treino e teste para colocar o modelo pra aprender sobre esses dados a fim de retornar a melhor predição possível em relação ao problema que quero encarar, e os modelos escolhidos para realizar esses testes foram:

- **DecisionTreeClassifier:** Cria uma ramificação de decisões para chegar no resultado final.
- **RandomForestClassifier:** Cria diversas ramificações de decisões para chegar no resultado final.
- **KNeighborsClassifier:** Se baseia nos valores próximos para chegar no resultado final.

Em cada um dos modelos apresentados, foram feitos diversos testes com mudanças nos hiperparâmetros, a fim de achar quais hiperparâmetros e modelo melhor satisfariam meu objetivo.

Para os modelos de árvore de decisão (DecisionTreeClassifier e RandomForestClassifier), foram aplicados os seguintes testes, “max_depth” com valores de 2, 3, 5 e 10, “criterion” para cada um dos valores anteriores alternando entre ‘gini’ e ‘entropy’, e por fim destaco que “random_state” foi fixado em 42 para melhor comparação nos resultados finais do modelo. Já para o modelo restante, foram aplicados os valores para “n_neighbors” de 2, 3, 5 e

10, totalizando em 20 a quantia final de testes que pode ser visualizado através das matrizes de confusão geradas no gráfico a seguir:



Dentre todas essas 20 matrizes de confusão, observe que em primeiro momento (com os hiperparâmetros no título de cada matriz) quem se sobressai com uma melhor precisão é justamente o modelo que ao final desses testes é o que tem a pior precisão, isso aconteceu visto que o KNN é muito suscetível a dados que possam desviar do verdadeiro padrão de base como um todo, então quanto mais “vizinhos” (n_neighbors) deixo esse modelo olhar, maior a probabilidade dele encontrar valores que não representam tão bem o geral dos dados. Já nos modelos de árvore de decisão, observe que a mudança do

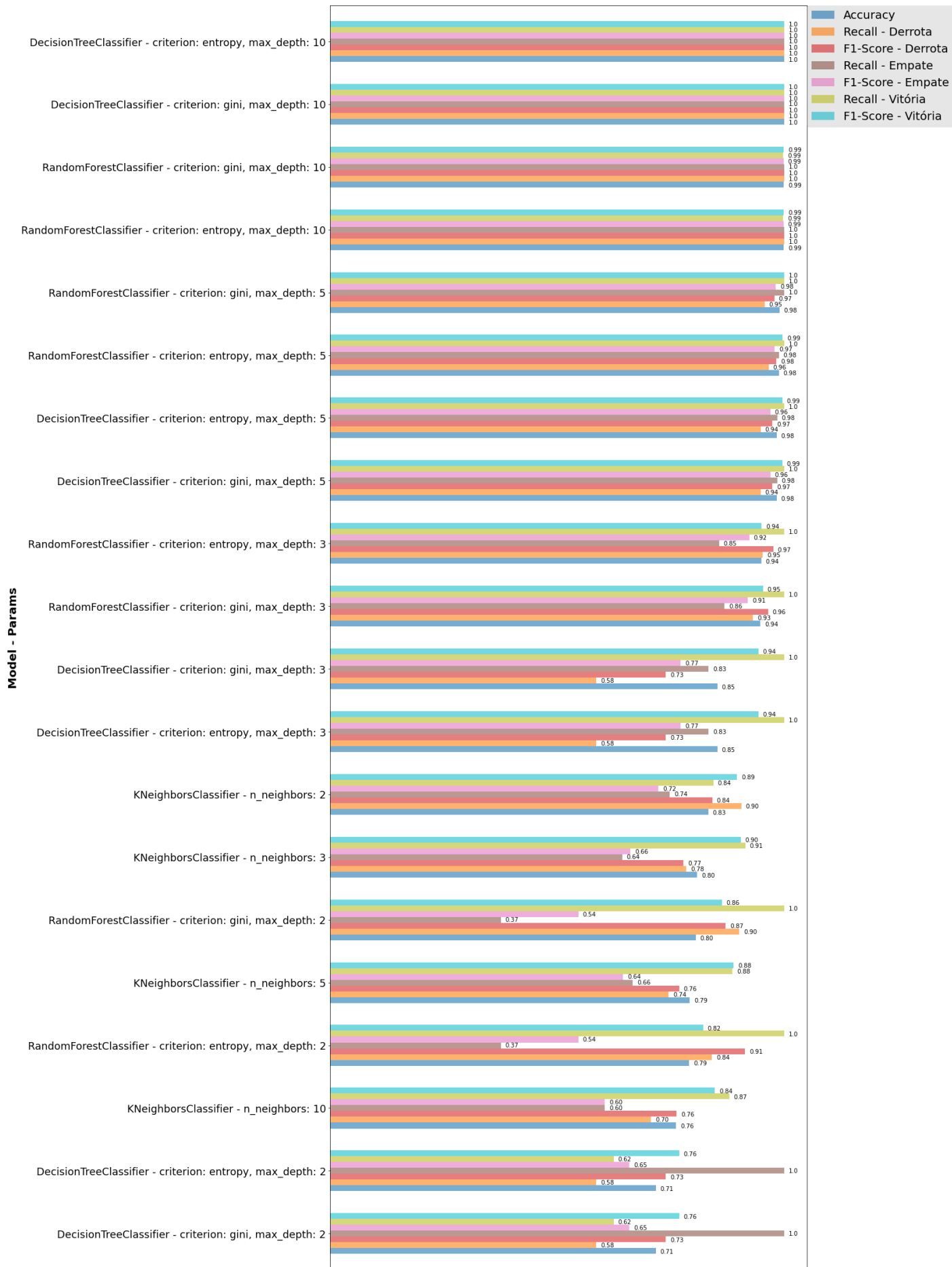
hiperparâmetro “criterion” não impacta significativamente os valores da precisão nesses modelos e que quanto mais aumento o valor de “max_depth”, mais preciso o modelo ficava, destaco também que as maiores mudanças na precisão foram entre os testes nas fileiras de imagem 1 a 3, já na quarta fileira, houve uma tendência de se estabilizar perto ao valor máximo possível da precisão.

Métricas de Desempenho

Ao decorrer de todos os testes realizados, foram coletados os dados que medem as métricas de desempenho para eles, tais como:

- **Accuracy:** Precisão geral do modelo.
- **Recall:** É o número de verdadeiros positivos e o número de falsos negativos. O recall é intuitivamente a habilidade do classificador de encontrar todas as amostras positivas.
- **F1-score:** A métrica F1 pode ser interpretada como uma média ponderada da precisão e recall.

Além dessas citadas, a precisão para cada uma das predições possíveis (Derrota, Empate e Vitória) também foi calculada. Após ter todos os dados relacionados às métricas, foi criado um gráfico que demonstra os desempenhos para cada um dos testes:



Pode-se observar em primeiro momento dois pontos, o modelo que melhor se saiu nas métricas de desempenho foi o “DecisionTreeClassifier” e o modelo que não conseguiu acompanhar na evolução dessas métricas, como o esperado, é o “KNeighborsClassifier”, e por fim, noto que as métricas começam a se estabilizar (nos modelos de árvore de decisão) a partir do hiperparâmetro “max_depth” igual a 5, com a accuracy chegando muito perto do valor máximo.

Implementação

Por fim de todo esse processo de separar as melhores características, os melhores dados, treinamento e métricas de modelos, vai ser realizado um teste, utilizando o modelo já treinado anteriormente, para prever o resultado de uma nova partida que adicionei manualmente, considerando somente os dez últimos jogos do time da casa para adicionar os valores fictícios.

É criada uma nova linha e após isso pego o modelo que obteve as melhores pontuações nas métricas calculadas para predição (DecisionTreeClassifier) desses meus testes, assim o modelo já treinado vai poder retornar o que ele vê como sendo a maior probabilidade de ocorrer (dentro os possíveis valores de Derrota, Empate e Vitória) nessa partida fictícia criada. Com essa predição feita e o valor de saída do modelo tratada para aparecer como um valor textual, o modelo retornou o seguinte resultado:

```
# Predizendo somente nova linha com o melhor modelo
y_pred_new_line = model_decision_tree.predict(df_new_row)
y_pred_trated = 'Derrota' if y_pred_new_line == 1 else 'Empate' if y_pred_new_line == 2 else 'Vitória'
print(f"Modelo apontou que para esse novo jogo, o resultado será de '{y_pred_trated}' para o time da casa")
```

Modelo apontou que para esse novo jogo, o resultado será de 'Vitória' para o time da casa

Observe que para essa específica linha criada, o modelo retornou, como sendo uma maior probabilidade, a resposta do time da casa tendo a “Vitória” sobre o time visitante.

Considerações Finais e Conclusão

Com o agente inteligente finalizado, coloco em foco dois pontos que me chamaram a atenção em determinadas situações no decorrer da construção dele, em primeiro, no momento de separação das melhores colunas para o modelo (feature engineering), pois a coluna “time_fora” não teve um impacto significativo ao prever a coluna target (resultado_casa), o que é no mínimo estranho (fica como um alerta de possível ponto de atenção para melhoria futura) e no segundo ponto, fiquei surpreso em ver o modelo que melhor performou, pois eu sabia que seria um dos modelos de árvore de decisão desde o início, porém diante dos diversos testes realizados, esperava que o

modelo de “RandomForestClassifier” se destacaria, mas o que pode ter acontecido, é de não ter dados tão significativamente diferentes para realmente esse modelo ficar em primeiro lugar.

Com relação às dificuldades que mais encontrei na realização desses modelos, foi a de achar alguma maneira dinâmica em transformar dados textuais em dados categóricos para as diferentes colunas (sem a utilização de “get_dummies”, da biblioteca Pandas) e também com esses diversos testes que eu estava para realizar, como faria para pegar as métricas de cada um dos testes e identificá-las depois para uma análise mais detalhada? Esse ponto eu resolvi criando uma lista (ao descobrir que consigo modificar a saída de retorno do “classification_report” para um dicionário), e adicionando todas as métricas identificadas de maneira correta de forma dinâmica para depois eu transformar em um dataset exclusivo com os valores dessas medidas.

Deixo aqui os possíveis pontos de melhorias que observei durante a realização do modelo:

1. **Aumentar o range de dados**, pois os dados utilizados foram apenas de um intervalo dez anos (2012 - 2021).
2. **Valores faltantes em diversas colunas** foi a principal causa da maioria das colunas serem retiradas antes e durante a realização de “feature engineering”.
3. **Mudança dos modelos** de classificação escolhidos podem ajudar o resultado final a ter uma considerável melhora.
4. **Povoar com mais colunas features** para que o modelo tenha uma visão mais ampla ao predizer o resultado final.
5. **Obrigação em ter dados** sobre partidas que ainda vão ocorrer, ou seja, não foram criados, a exemplo das colunas “placar_casa” e “placar_fora”, para o modelo realizar a predição. Isso poderia ser resolvido, pensando de forma breve, fazendo a criação de um segundo modelo que alimentaria esse primeiro com dados fictícios mais firmemente baseados nos reais.

Evolui muito na realização desse modelo, era algo que já estava querendo há algum tempo, pois já tinha uma base de dados que eu mesmo estruturei e o intuito era realmente esse, de aplicar conhecimento em cima desses dados, provável que esse foi um bom início de algumas das diversas melhorias que ainda estão por vir.

Referências Bibliográficas

UFMG - Universidade Federal de Minas Gerais. Introdução à Análise de Dados em Saúde com Python: Introdução ao Python para análise de dados. Belo Horizonte, MG: Universidade Federal de Minas Gerais, 2023. Disponível em: <https://www.bu.ufmg.br/imagem/00002c/00002cd9.pdf>. Acesso em: 20 nov. 2024.

scikit-learn developers (BSD License). scikit-learn user guide: sklearn.metrics: Metrics. 30 jul. 2019. Disponível em: https://scikit-learn.org/0.20/_downloads/scikit-learn-docs.pdf. Acesso em: 09 dez. 2024.